Intelligent Analyst Verification Protocol (IAVP)

Public Verification Specification

Document ID	IA-VP-1.0
Version	1.0
Publication Date	2026-02-18
Status	Final
Owner	Intelligent Analyst, Inc.

1. Scope

This specification defines the Intelligent Analyst Verification Protocol (IAVP) version 1.0. IAVP provides a method for independent verification of attestation artifacts produced by Intelligent Analyst processing batches.

IAVP v1.0 defines batch-level attestation only. Record-level attestation is not within the scope of this version.

This specification is intended for use by auditors, compliance officers, and technical personnel who require independent verification of processing results without access to the Intelligent Analyst platform.

2. Definitions

Attestation Manifest: A JSON document containing metadata, metrics, and cryptographic commitments.

Evidence Pack: An artifact bundle containing processing results and attestation materials.

Hash Chain: A cryptographic structure linking ordered records through iterative hashing.

Root Hash: The final hash value of a hash chain, serving as a commitment to the entire sequence.

JCS: JSON Canonicalization Scheme as defined in RFC 8785.

3. Canonical Serialization

All JSON documents subject to hashing or signing MUST be serialized according to RFC 8785 (JSON Canonicalization Scheme). JCS requirements: object keys MUST be sorted lexicographically by UTF-16 code units; no whitespace between tokens; numbers MUST use shortest representation; strings MUST use minimal escape sequences.

4. Record Hashing

Each record MUST be hashed using SHA-256. The hash input is the JCS-canonical byte representation of the record JSON object. The record_hash MUST be a 64-character lowercase hexadecimal string.

5. STABLE_INPUT_ORDER_V1

Before constructing the hash chain, records MUST be ordered according to STABLE_INPUT_ORDER_V1:

> Normalization: (1) Timestamps to RFC 3339 UTC with 6 fractional digits; (2) source_system_id to Unicode NFC; (3) record_hash as lowercase hex.
>
> Sort Order: Records sorted ascending by (A) source_timestamp, (B) source_system_id (UTF-8 byte lexicographic), (C) record_hash.

6. Hash Chain Construction (SHA256_CHAIN_V1)

Genesis hash H[0] = 32 bytes of zero. For i=1 to N: H[i] = SHA256(bytes(H[i-1]) || bytes(record_hash[i])). Root hash = H[N]. bytes(x) converts a 64-char hex string to 32-byte binary.

7. Attestation Manifest Schema

Required fields: protocol_version ("IA-VP-1.0"), artifact_type ("EVIDENCE_PACK" | "STRESS_TEST_CERT"), artifact_version, artifact_mode ("DEMO_SIMULATED" | "PRODUCTION_REAL"), generated_at_utc, engine_version, config_hash_sha256, batch_id, dataset_hash_sha256, hash_chain (method, root_hash_sha256, record_count, ordering), metrics (l1_pct, l2_pct, l3_pct, l4_pct, replay_variance=0), key (key_id, pubkey_fingerprint_sha256).

8. Signature Specification

Algorithm: ECDSA P-256 (prime256v1 / secp256r1). Hash: SHA-256. Encoding: DER. Procedure: manifest_bytes = JCS(manifest_json); manifest_hash = SHA256(manifest_bytes); signature_der = ECDSA_P256_SIGN(private_key, manifest_hash). Public key fingerprint: SHA256(SubjectPublicKeyInfo DER).

9. Public Verification Procedure

Required files: public_key.pem, attestation_manifest.jcs.json, attestation_signature.der

Verification command:

```
openssl dgst -sha256 -verify public_key.pem -signature attestation_signature.der
attestation_manifest.jcs.json
```

Expected output: `Verified OK`

Optional fingerprint verification:

```
openssl pkey -pubin -in public_key.pem -outform DER | openssl dgst -sha256
```

10. Versioning Policy

- v1.0 is immutable. No silent edits SHALL be made.
- Clarifications MAY be issued as v1.0.x (non-breaking).
- Breaking changes require v2.0.
- Historical versions SHALL remain accessible at original URLs.

Last updated: 2026-02-18